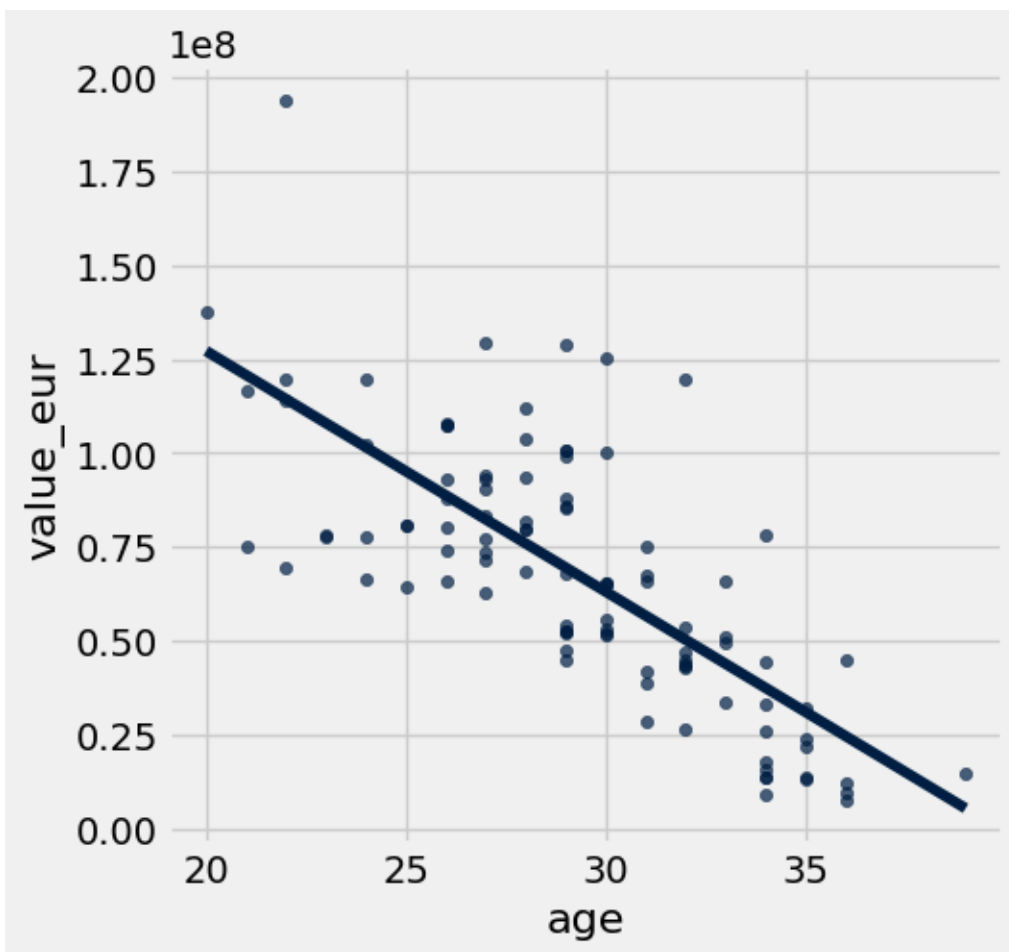

Question 2.1. Before jumping into any statistical techniques, it's important to see what the data looks like, because data visualizations allow us to uncover patterns in our data that would have otherwise been much more difficult to see. **(3 points)**

Create a scatter plot with age on the x-axis ("age"), and the player's value in Euros ("value_eur") on the y-axis.

```
In [22]: fifa.scatter("age", "value_eur", fit_line=True)
```

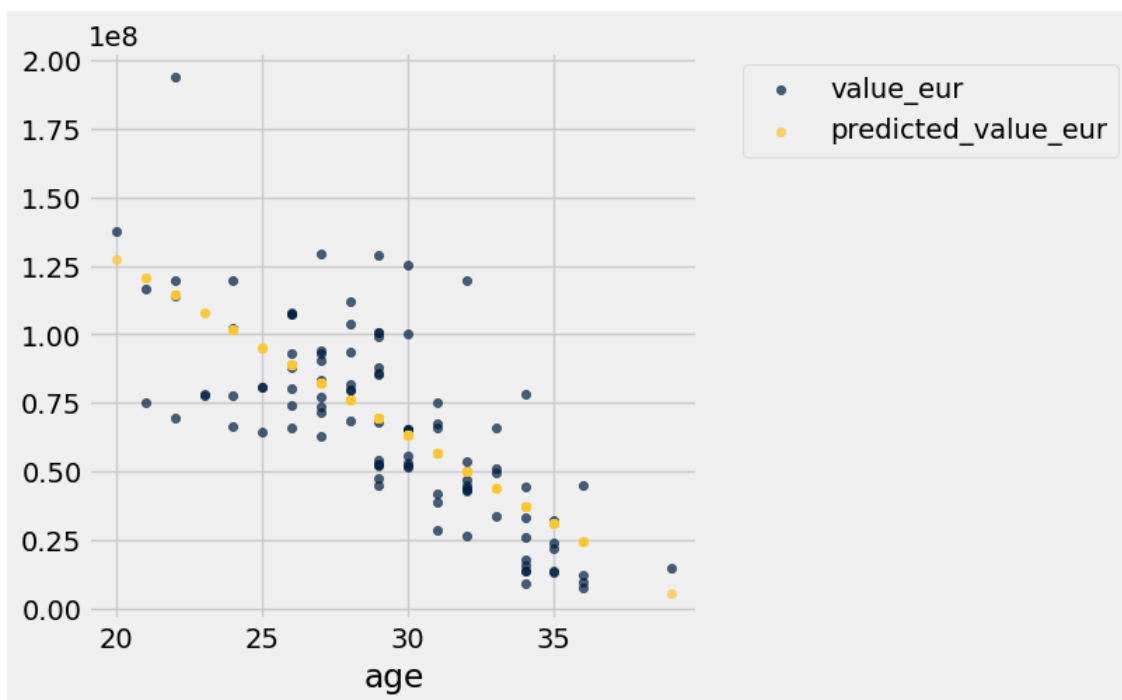


Question 2.3. Create a scatter plot with player age (“age”) along the x-axis and both real player value (“value_eur”) and predicted player value along the y-axis. The predictions should be created using a fitted **regression line**. The color of the dots for the real player values should be different from the color for the predicted player values. **(8 points)**

Hint 1: Feel free to use functions you have defined previously.

Hint 2: [15.2](#) and [7.3](#) has examples of creating such scatter plots.

```
In [26]: predictions = predict(fifa, "age", "value_eur")
         fifa_with_predictions = fifa.with_columns("predicted_value_eur", predictions)
         fifa_with_predictions.scatter("age", ["value_eur", "predicted_value_eur"])
```



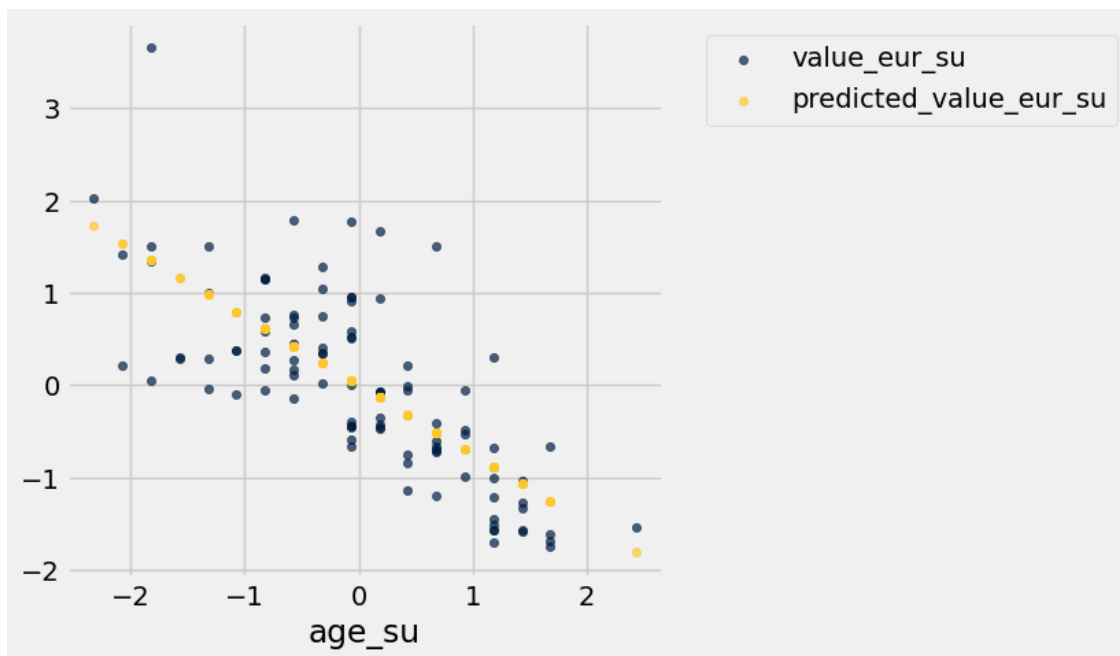
Question 2.4. Looking at the scatter plot you produced above, is linear regression a good model to use? If so, what features or characteristics make this model reasonable? If not, what features or characteristics make it unreasonable? **(5 points)**

Looking at the scatter plot, linear regression seems like a good fit. The data points form a nearly straight line, which means there's a clear relationship between the variables. The points are not too spread out and don't show any weird curves or sudden jumps. This suggests that a simple straight-line model can explain how the variables are connected. The points go up or down in a predictable way, making linear regression a reasonable choice for understanding the pattern in this data.

Question 2.5. In 2.3, we created a scatter plot **in original units**. Now, create a scatter plot with player age **in standard units** along the x-axis and both real and predicted player value **in standard units** along the y-axis. The color of the dots of the real and predicted values should be different. (8 points)

Hint: Feel free to use functions you have defined previously.

```
In [27]: age_su = standard_units(fifa.column("age"))
value_su = standard_units(fifa.column("value_eur"))
predictions_su = predict(Table().with_columns("age", age_su, "value_eur", value_su), "age", "value_eur")
fifa_su = Table().with_columns("age_su", age_su, "value_eur_su", value_su, "predicted_value_eur_su", predictions_su)
fifa_su.scatter("age_su", ["value_eur_su", "predicted_value_eur_su"])
```



Question 2.6. Compare your plots in 2.3 and 2.5. What similarities do they share? What differences do they have? (5 points)

In comparing the plots from sections 2.3 and 2.5, both share a linear trend showing player value increasing with age. The key difference is scaling: section 2.3 uses raw data, while section 2.5 presents standardized data normalized to a standard scale (approximately -2 to 3). Standardization in 2.5 removes unit-specific influences, making the underlying data relationship more statistically interpretable and suitable for machine learning analysis.

Question 2.8. Use the `rmse` function you defined along with `minimize` to find the least-squares regression parameters predicting player value from player age. Here's an [example](#) of using the `minimize` function from the textbook. (10 points)

Then set `lsq_slope` and `lsq_intercept` to be the least-squares regression line slope and intercept, respectively.

Finally, create a scatter plot like you did in 2.3 with player age (“age”) along the x-axis and both real player value (“value_eur”) and predicted player value along the y-axis. **Be sure to use your least-squares regression line to compute the predicted values.** The color of the dots for the real player values should be different from the color for the predicted player values.

Note: Your solution should not make any calls to the slope or intercept functions defined earlier.

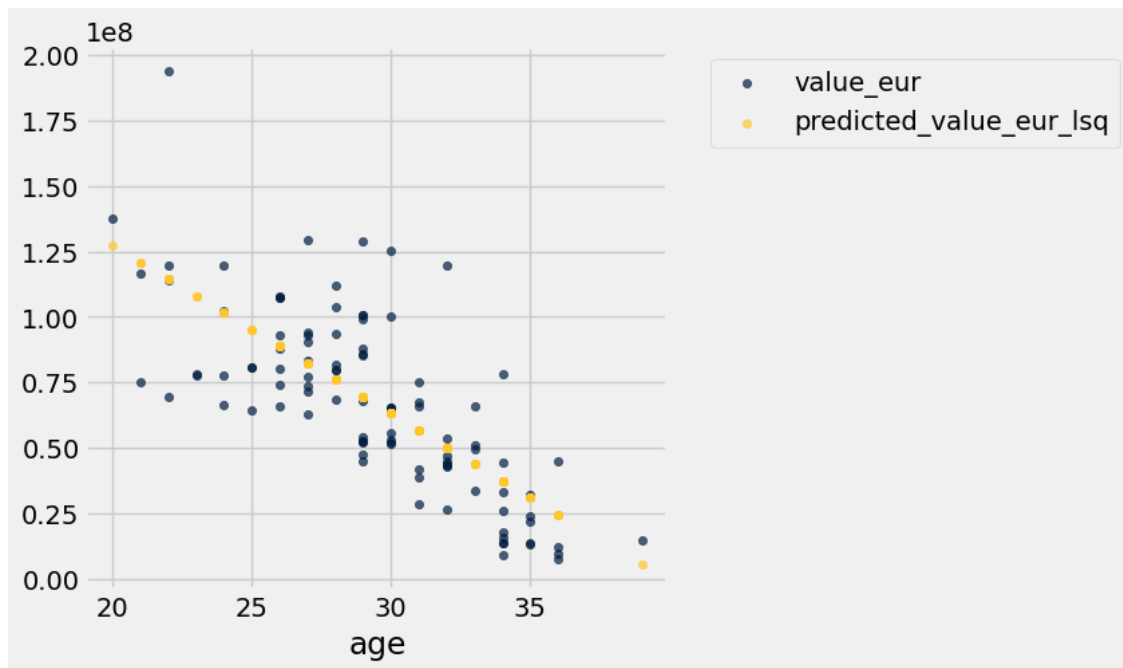
Hint: Your call to `minimize` will return an array of argument values that minimize the return value of the function passed to `minimize`.

```
In [36]: minimized_parameters = minimize(rmse)
         lsq_slope = minimized_parameters.item(0)
         lsq_intercept = minimized_parameters.item(1)

         # This just prints your slope and intercept
         print("Slope: {:.g} | Intercept: {:.g}".format(lsq_slope, lsq_intercept))

         fifa_with_lsq_predictions = fifa.with_columns("predicted_value_eur_lsq", lsq_slope * fifa.columns["age"] + lsq_intercept)
         fifa_with_lsq_predictions.scatter("age", ["value_eur", "predicted_value_eur_lsq"])
```

Slope: -6.41462e+06 | Intercept: 2.55525e+08



Question 2.9. The resulting line you found in 2.8 should appear very similar to the line you found in 2.3. Why were we able to minimize RMSE to find nearly the same slope and intercept from the previous formulas? **(5 points)**

Hint: Re-reading [15.3](#) might be helpful here.

The regression line formulas are designed to minimize RMSE, which is why both analytical and numerical methods arrive at the same solution.

